DLSCMS User Manual

Defence Logistics and Supply Chain Management System — Terminal Edition Authors: Vedant Pathak (R045), Dhairya Shah (R057), Kanishak Thakran (R046) · SVKM's NMIMS MPSTME, Mumbai Version 1.0 · *Status: verified against the Phase 3 build; every example below was run with `--script`. Screen captures are refreshed in Phase 8.*

---

1. What this program does

DLSCMS helps logistics staff keep several bases stocked. You can register bases, see what each base holds, raise supply requests, let the system find a base with spare stock, assign a suitable vehicle, dispatch the delivery, and watch inventory update when it arrives. All data is **synthetic** (made up for demonstration); no real operational data is used.

2. What you need

Item	Requirement
Java	JDK/JRE **17 or newer** (free: Eclipse Temurin from adoptium.net). Check with `java -version`.
Terminal	**Windows Terminal** (best), macOS Terminal, or any Linux terminal, at least 100 columns × 30 rows. Old `cmd.exe` windows may show garbled colours — use `--plain` (section 9).
Disk	< 5 MB. No internet, no installation, no database.

3. Starting the program

Open a terminal in the project folder (`OOP\dlscms`). | Goal | Windows | Linux / macOS | |—| —|—| | Run (default `small` dataset) | `run.bat` | `./run.sh` | | Run without colours | `run.bat --plain` | `./run.sh --plain` | | Pick a dataset | `run.bat --data data\medium` | `./run.sh --data data/medium` | | Run the scripted demo and exit | `run.bat --demo` | `./run.sh --demo` | | Run a command file | `run.bat --script cmds.txt` | `./run.sh --script cmds.txt` | | Build from source (needs JDK) | `scripts\build.bat test` | `scripts/build.sh test` | | One simulation run (replays `scenario_requests.csv`, prints one CSV row) | `run.bat --sim --data data\medium --src BALANCED --veh FASTEST` | `./run.sh --sim --data data/medium --src BALANCED --veh FASTEST` | Without the helper scripts: `java -jar dist/dlscms.jar [options]`.

4. Reading the screen

```
 DLSCMS ▸ DEFENCE LOGISTICS TERMINAL     [MON]     DAY 03 14:00
SRC:NEAREST  VEH:FASTEST   ← header bar

_____________________________________________________________________
_____________________________

  (body: tables and panels for the current screen)
← body

_____________________________________________________________________
_____________________________

  ALERT ▸ PUN: MED-KIT below reorder level (40 < 120)
← status line
 DLSCMS ▸ _
← command prompt
```

- **Header:** product name, current screen code, the **simulated clock** (`SIM DAY 03 14:00` = day 3, 14:00) and the two active strategies. The clock starts at `DAY 00 00:00` and moves **only** when you type `TICK <hours>` (or when a loaded file already had a later time); creating, planning and dispatching take no simulated time. A * at the end of the header means unsaved changes.
- **Status line:** the latest alert or message. **Red `ERR ▸ …`** means your last command was refused and nothing changed.
- **Prompt:** type a *function code* (like `INV`) plus optional arguments and press **Enter**. Codes are not case-sensitive. Press **Enter on an empty line** to refresh. Type `MON` to return home. Type `?` after any code (e.g. `REQ ?`) for its usage.
- **Colours:** amber = normal text · cyan = ids you can type · green = healthy/delivered · red = shortage/failed/error · grey = borders.

5. Quick start: the “Pune needs medical kits” tutorial (about 3 minutes)

1. Start the program and look at the dashboard (`MON`). Pune shows a red **MED-KIT** shortage.
2. Check Pune’s stock: `INV PUN`.
3. Raise a request for 200 kits: `REQ NEW PUN MED-KIT 200 URGENT` → the system answers with a new id such as `R-0001`.
4. Approve it: `REQ APPROVE R-0001`.
5. Let the system plan the supply: `REQ FULFIL R-0001`. It finds bases with spare kits, picks a vehicle for each trip, and creates deliveries (e.g. `D-0001`). Stock is *reserved* but has not left yet.
6. Look at the plan: `DLV` (list) then `DLV D-0001` (detail: route, distance, vehicle, ETA).
7. Send it: `DLV DISPATCH D-0001` — the donor base’s stock drops and the vehicle is now in transit.
8. Move time forward: `TICK 12` — deliveries whose ETA has passed arrive automatically, Pune’s stock rises, and the vehicle becomes available again at Pune.
9. See the result: `INV PUN`, then `RPT KPI`.

10. Save your work: `SAVE`. Shortcut: type `DEMO` to watch these steps run automatically; press Enter between steps.

6. Command reference

Arguments in `<angle brackets>` are required, `[square brackets]` optional. Ids (`PUN`, `MED-KIT`, `R-0001`) are not case-sensitive.

6.1 Bases (F1) — `BASE`

Command	What it does
`BASE`	List all bases with location and storage use
`BASE <id>`	Details of one base
`BASE ADD <id> <name> <lat> <lon> [capacity]`	Register a new base (use _ for spaces in the name)
`BASE REMOVE <id>`	Remove a base — refused if it still holds stock or has open deliveries; asks to confirm

Example: `BASE ADD SRI Srinagar_Node 34.08 74.80 4000`

6.2 Inventory (F2) — `INV`, `SUP`

Command	What it does
`SUP`	Supply catalogue (id, name, category, unit weight)
`INV`	Network-wide stock summary, shortages first
`INV <base>`	Stock ledger of one base: quantity, reserved, available, reorder level, status (▼ SHORT / ▲ OK)
`INV ADJ <base> <supply> <±qty>`	Add or remove stock, e.g. `INV ADJ PUN MED-KIT -15` (cannot go below zero or below reserved)
`INV SET <base> <supply> <qty> [reorder]`	Set stock (and optionally the reorder level)

Reserved = promised to a planned delivery. *Available* = quantity − reserved. *Surplus* = available above the reorder level — only surplus can be given to other bases.

6.3 Requests (F3) — `REQ`

Command	What it does
`REQ`	Request book sorted by priority, then age
`REQ <id>`	Request detail and its deliveries
`REQ NEW <base> <supply> <qty> [ROUTINE|URGENT|CRITICAL]`	Create a request (default priority ROUTINE)
`REQ APPROVE <id>` / `REQ REJECT`	Move the request along; invalid moves are

Command	What it does
`<id>` / `REQ CANCEL <id>`	refused with a message
`REQ FULFIL <id>`	Plan deliveries automatically (may create several for a big request)

Priority rules: CRITICAL requests are served first and may draw donor bases down to half their reorder level; ROUTINE only uses true surplus. Request states: NEW → APPROVED → FULFILLING → FULFILLED (or REJECTED / CANCELLED).

6.4 Fleet (F4) — `FLT`

Command	What it does
`FLT`	All vehicles with type, base, capacity, status
`FLT AVAIL [category]`	Only available vehicles, optionally those able to carry MEDICAL / FOOD / FUEL / SPARES / SHELTER
`FLT <id>`	One vehicle's details
`FLT MAINT <id> ON|OFF`	Take a vehicle out of / back into service

Vehicle types: **Truck** (general, no fuel), **FuelTruck** (fuel only), **MedicalVan** (medical and food, fast), **Helicopter** (fast, small loads, no fuel).

6.5 Deliveries (F4, F5) — `DLV`, `TICK`

Command	What it does
`DLV` / `DLV <id>`	List deliveries / one delivery with its timeline
`DLV PLAN <req> [VEH <vehicle>]`	Plan one delivery for a request; name a vehicle to override the automatic choice
`DLV DISPATCH <id>` / `DLV DISPATCH ALL`	Start the trip (or every planned trip) (PLANNED → IN_TRANSIT); stock leaves the donor base
`DLV COMPLETE <id>`	Mark arrived now (IN_TRANSIT → DELIVERED); stock added to the destination
`DLV FAIL <id> [reason]`	Mark failed; reserved or in-transit stock goes back to the donor
`TICK <hours>`	Advance the clock; deliveries whose ETA has passed complete automatically

6.6 Reports (F6) — `RPT`

Command	Shows
`RPT STOCK`	Stock by base and supply
`RPT SHORT`	Shortages and surpluses across the network
`RPT DLV`	Deliveries by status and vehicle
`RPT KPI`	Fulfilment rate, average lead time by priority, vehicle utilisation, delivery counts, shortage

Command	Shows
	cells
`RPT <kind> EXPORT`	Also writes the report as CSV into the `reports/` folder (e.g. `reports/kpi.csv`)

6.7 Strategies — STRAT

Command	Meaning
`STRAT`	Show the active strategies
`STRAT SRC NEAREST`	Take stock from the nearest base with surplus (fast delivery)
`STRAT SRC SURPLUS`	Take from the base with the most surplus (protects small donors)
`STRAT SRC BALANCED`	Mix of closeness and surplus
`STRAT VEH SMALLEST`	Smallest vehicle that fits the load (saves big vehicles)
`STRAT VEH FASTEST`	Vehicle that would arrive soonest
`AUTO ON` / `AUTO OFF`	Dispatch deliveries automatically as soon as they are planned (default OFF)

6.8 Saving, loading, leaving

Command	What it does
`SAVE [folder]`	Write all data as CSV files (default: the folder you started with)
`LOAD [folder]`	Replace current data from CSV files — asks to confirm
`DEMO`	Scripted walkthrough of the Pune scenario
`HELP`	Command list
`EXIT` / `QUIT`	Leave (offers to save if there are unsaved changes)

7. Data files

Each dataset folder (`data/small`, `data/medium`, `data/large`) holds plain CSV files you can open in Excel: `bases.csv`, `supplies.csv`, `inventory.csv`, `vehicles.csv`, `requests.csv`, `deliveries.csv`, `meta.csv` (clock and counters), plus `scenario_requests.csv` (request stream used by the simulator, not loaded by the terminal). Column definitions are in `docs/dataset.md`. Edit them with the program closed.

8. Worked example of a failure and how the system reacts

You type	Response	Why
`REQ FULFIL R-9999`	`ERR ▸ Unknown request id R-9999`	invalid id

You type	Response	Why
`INV ADJ PUN MED-KIT -9999`	`ERR ▸ Insufficient stock: PUN has 40 MED-KIT available, cannot remove 9999`	would go below zero
`DLV DISPATCH D-0001` twice	`ERR ▸ Delivery D-0001 is IN_TRANSIT; cannot dispatch`	invalid state change
`BASE REMOVE PUN`	`ERR ▸ Cannot remove PUN: it still holds stock (428 BLANKET)`	bases must be empty
`REQ FULFIL` with every capable vehicle busy or in maintenance	`ERR ▸ No vehicle available for the supply MED-KIT (try TICK, FLT AVAIL, or free a vehicle)`	take vehicles out of maintenance or wait for trips to finish

Nothing changes when an error is shown, so it is always safe to retry.

9. Troubleshooting

Symptom	Fix
`'java' is not recognized`	Install JDK 17+ and reopen the terminal (check `java -version`).
`UnsupportedClassVersionError`	Your Java is older than 17 — install a newer JDK.
Strange characters like `←[33m`	Terminal does not support colours: run with `--plain`, or use Windows Terminal.
Box lines look broken	Use a UTF-8 font/terminal, or `--plain` (ASCII borders).
Screen too small / text wraps	Enlarge the window to at least 100 × 30.
`ERR ▸ data format error in inventory.csv line 12`	The CSV was edited incorrectly; fix that line or regenerate the dataset (`python3 scripts/gen_data.py`).

10. Glossary

Base – a logistics node holding stock · **Supply** – an item type in the catalogue · **Reorder level** – the stock level below which a base counts as short · **Reserved** – stock promised to a planned delivery · **Surplus** – available stock above the reorder level · **Delivery** – one trip of one vehicle carrying one supply from one base to another · **ETA** – simulated arrival time · **Tick** – advancing the simulated clock · **Strategy** – the rule used to choose a donor base or a vehicle.